Compte Rendu

TP 2 — Analyse Lexicale avec **Flex**

Techniques de Compilation • ENSA de Fès

Année Universitaire 2025/2026 • 6 avril 2026

Objectifs du TP

1. Comprendre le rôle de l'analyse lexicale dans la chaîne de compilation.
2. Maîtriser la structure d'un fichier Flex (`.l`) : trois sections.
3. Écrire des expressions régulières pour reconnaître des tokens.
4. Utiliser `yytext`, `yyleng` et la fonction `yylex()`.
5. Compiler et exécuter : `flex` → `gcc` → exécution.

0 Contents

1 Introduction **2**
1.1 Mise en place de l'environnement 2

2 Partie 1 : Compteur de mots **4**
2.1 Objectif 4
2.2 Code source — `partie1.l` 4
2.3 Compilation et test 5
2.4 Réponses aux questions 6

3 Partie 2 : Identifiants et mots-clés **6**
3.1 Objectif 6
3.2 Code source — `partie2.l` 6
3.3 Compilation et test 7
3.4 Réponses aux questions 8

4 Partie 3 : Nombres et opérateurs **8**
4.1 Objectif 8
4.2 Code source — `partie3.l` 8
4.3 Compilation et test 9
4.4 Réponses aux questions 10

5 Partie 4 : Mini-projet — Analyseur MiniLang **11**
5.1 Spécification du langage 11
5.2 Code source — `minilang.l` 11
5.3 Fichier de test — `test.ml` 13
5.4 Résultat de l'analyse 14
5.5 Tokens de `float res = (pi * 2.0) + x;` 16
5.6 Réponses aux questions 16

6 Conclusion **17**

Annexe — Récapitulatif **18**

1 Introduction

L'analyse lexicale est la **première phase** d'un compilateur. Elle lit le code source caractère par caractère et produit une suite de **tokens**. **Flex** automatise cette construction à partir d'un fichier `.l` structuré en **trois sections** séparées par `%%` :

1. **Section 1 — Déclarations :** `%{...%}`, définitions nommées, options.
2. **Section 2 — Règles :** paires *motif* `{ action C }`.
3. **Section 3 — Code utilisateur :** fonctions C (`main`, etc.).

Élément	Rôle
`yytext`	Pointeur `char*` vers le lexème reconnu
`yyleng`	Longueur (en caractères) du lexème
`yylex()`	Lance l'analyseur lexical
`yywrap()`	Appelée en fin de fichier (retourner 1)
`ECHO`	Réécrit `yytext` sur la sortie standard

1.1 Mise en place de l'environnement

Avant de commencer les exercices, nous vérifions l'installation de `gcc` et de `flex` sous Windows (GnuWin32), puis nous testons un premier analyseur `bonjour.l` en mode interactif.

Capture 1 - Vérification de l'environnement & premier test

```
Invite de commandes - bonjo

Microsoft Windows [version 10.0.26200.8037]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\pcc>gcc --version
gcc (tdm64-1) 9.2.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.


C:\Users\pcc>cd C:\Users\pcc\Downloads\flex

C:\Users\pcc\Downloads\flex>flex bonjour.l

C:\Users\pcc\Downloads\flex>gcc lex.yy.c -o bonjour.exe

C:\Users\pcc\Downloads\flex>bonjour.exe
=== Analyse en cours ===

TP2
Mot n 1 : TP
Nombre : 2
```

Capture 2 - Exécution interactive de bonjour.exe

```
C:\Users\pcc\Downloads\flex>bonjour.exe
=== Analyse en cours ===

echo "Bonjour le monde 2025 " |  ./bonjour
Mot n 1 : echo
Mot n 2 : Bonjour
Mot n 3 : le
Mot n 4 : monde
Nombre : 2025
Mot n 5 : bonjour
^Z

=== Total : 5 mot(s) trouve(s) ===
```

Capture 3 - Test avec redirection de fichier (texte.txt)

```
C:\Users\pcc\Downloads\flex>echo "Le compilateur Flex analyse les tokens." > texte.txt

C:\Users\pcc\Downloads\flex>bonjour.exe < texte.txt
=== Analyse en cours ===

Mot n 1 : Le
Mot n 2 : compilateur
Mot n 3 : Flex
Mot n 4 : analyse
Mot n 5 : les
Mot n 6 : tokens

=== Total : 6 mot(s) trouve(s) ===

C:\Users\pcc\Downloads\flex>
```

2 Partie 1 : Compteur de mots

2.1 Objectif

Écrire un analyseur qui compte les **mots**, **lignes** et **caractères** d'un texte (comportement similaire à `wc`).

2.2 Code source — partie1.l

partie1.l

```
%{
#include <stdio.h>
int nb_mots   = 0;
int nb_lignes = 0;
int nb_chars  = 0;
%}

%option noyywrap

%%

/* Sauts de ligne */
\n            { nb_lignes++; nb_chars++; }

/* Mots : incrementer nb_mots, ajouter yyleng a nb_chars */
[^ \t\n]+     { nb_mots++; nb_chars += yyleng; }

/* Tout autre caractere */
.             { nb_chars++; }

%%

int main(void) {
    yylex();
    printf("Lignes      : %d\n", nb_lignes);
    printf("Mots        : %d\n", nb_mots);
```

```
    printf("Caracteres : %d\n", nb_chars);
    return 0;
}
```

2.3 Compilation et test

Capture 4 - Compilation de partie1.l

```
C:\Users\pcc\Downloads\flex>flex partie1.l

C:\Users\pcc\Downloads\flex>
C:\Users\pcc\Downloads\flex>gcc lex.yy.c -o partie1
```

Capture 5 - Exécution : echo "bonjour le monde ! " | ./partie1

```
PS C:\WINDOWS\System32> cd C:\Users\pcc\Downloads\flex
PS C:\Users\pcc\Downloads\flex> echo "bonjour le monde ! " | ./partie1
Lignes : 1
Mots : 4
Caracteres : 20
PS C:\Users\pcc\Downloads\flex>
```

Sortie observée

```
$ flex partie1.l && gcc lex.yy.c -o partie1
$ echo "bonjour le monde ! " | ./partie1

Lignes     : 1
Mots       : 4
Caracteres : 20
```

2.4 Réponses aux questions

? Questions

1. `%option noyywrap` : Supprime l'obligation de définir `yywrap()`. Sans elle, la compilation échoue (sauf si on lie `-lfl`). Avec `noyywrap`, le code généré est autonome et portable.
2. `yytext` vs `yyleng` : `yytext` est un pointeur vers la chaîne reconnue (ex. `"Bonjour"`), `yyleng` est sa longueur entière (ex. 7).
3. `flex partie1.l` **produit :** le fichier `lex.yy.c` contenant l'automate fini déterministe (AFD) correspondant aux règles définies.
4. **Ordre des règles :** Flex choisit la *plus longue correspondance* ; à longueur égale, la *première règle déclarée* est prioritaire.

3 Partie 2 : Identifiants et mots-clés

3.1 Objectif

Distinguer les **mots-clés** C (`int`, `if`, `else`, `while`, `for`, `return`) des **identifiants**.

3.2 Code source — `partie2.l`

partie2.l

```
%{
#include <stdio.h>
%}

%option noyywrap

LETTRE  [a-zA-Z_]
CHIFFRE [0-9]
IDENT   {LETTRE}({LETTRE}|{CHIFFRE})*

%%

/* Mots-cles AVANT les identifiants */
"int"      { printf("MOT-CLE  : %s\n", yytext); }
"if"       { printf("MOT-CLE  : %s\n", yytext); }
"else"     { printf("MOT-CLE  : %s\n", yytext); }
"while"    { printf("MOT-CLE  : %s\n", yytext); }
"for"      { printf("MOT-CLE  : %s\n", yytext); }
"return"   { printf("MOT-CLE  : %s\n", yytext); }

/* Identifiants */
{IDENT}    { printf("IDENT    : %s\n", yytext); }

/* Entiers */
[0-9]+     { printf("ENTIER   : %s\n", yytext); }
```

```
/* Ignorer espaces et tabulations */
[ \t\n]+   { /* ignorer */ }

/* Caracteres inconnus */
.          { printf("INCONNU  : %s\n", yytext); }

%%

int main(void) {
    yylex();
    return 0;
}
```

3.3 Compilation et test

Capture 6 - Compilation de partie2.l

```
C:\Users\pcc\Downloads\flex>flex partie2.l

C:\Users\pcc\Downloads\flex>gcc lex.yy.c -o partie2

C:\Users\pcc\Downloads\flex>
```

Capture 7 - Exécution : echo "int main if x1 return _var" | ./partie2

```
PS C:\Users\pcc\Downloads\flex> echo "int main if x1 return _var" | ./partie2
MOT-CLE : int
IDENT : main
MOT-CLE : if
IDENT : x1
MOT-CLE : return
IDENT : _var
PS C:\Users\pcc\Downloads\flex>
PS C:\Users\pcc\Downloads\flex>
```

Sortie observée

```
$ echo "int main if x1 return _var" | ./partie2

MOT-CLE  : int
IDENT    : main
MOT-CLE  : if
IDENT    : x1
```

```
MOT-CLE   : return
IDENT     : _var
```

3.4 Réponses aux questions

? Questions

1. **Mots-clés avant identifiants :** En cas d'égalité de longueur, Flex applique la première règle. Si `{IDENT}` était en premier, `int` serait reconnu comme identifiant.
2. **Équivalence avec `{IDENT}` :** `[a-zA-Z_]([a-zA-Z0-9_]*)` est strictement équivalente : les définitions nommées sont de simples alias d'expressions régulières.
3. **Chaînes de caractères :** L'expression `\"([^\"\n])*\"` reconnaît une chaîne entre guillemets sans guillemet interne ni saut de ligne.

4 Partie 3 : Nombres et opérateurs

4.1 Objectif

Reconnaître des **entiers**, **flottants**, **opérateurs** arithmétiques/relationnels et **délimiteurs**.

4.2 Code source — partie3.l

partie3.l

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

%option noyywrap

CHIFFRE  [0-9]
ENTIER   [+-]?{CHIFFRE}+
FLOTTANT [+-]?{CHIFFRE}+"."{CHIFFRE}+

%%

/* Flottant AVANT entier */
{FLOTTANT}   { printf("FLOTTANT  : %s (valeur : %.4f)\n",
                       yytext, atof(yytext)); }
{ENTIER}     { printf("ENTIER    : %s (valeur : %d)\n",
                       yytext, atoi(yytext)); }

/* Operateurs deux caracteres en premier */
```

```
"=="          { printf("OPERATEUR : %s\n", yytext); }
"!="          { printf("OPERATEUR : %s\n", yytext); }
"<="          { printf("OPERATEUR : %s\n", yytext); }
">="          { printf("OPERATEUR : %s\n", yytext); }

/* Operateurs un caractere */
[+\-*/%=<>]   { printf("OPERATEUR : %s\n", yytext); }

/* Delimiteurs */
[(){};,]      { printf("DELIM     : %s\n", yytext); }

/* Commentaires de ligne */
"//".*        { /* ignorer */ }

/* Espaces */
[ \t\n]+      { /* ignorer */ }

/* Caracteres inconnus */
.             { printf("INCONNU   : %s\n", yytext); }

%%

int main(void) {
    yylex();
    return 0;
}
```

4.3 Compilation et test

Capture 8 - Compilation de partie3.l

```
C:\Users\pcc\Downloads\flex>flex partie3.l

C:\Users\pcc\Downloads\flex>gcc lex.yy.c -o partie3

C:\Users\pcc\Downloads\flex>
```

Capture 9 - Exécution : echo "int main if x1 return _var" | ./partie3

```
PS C:\Users\pcc\Downloads\flex>
PS C:\Users\pcc\Downloads\flex> echo "int main if x1 return _var" | ./partie3
INCONNU : i
INCONNU : n
INCONNU : t
INCONNU : m
INCONNU : a
INCONNU : i
INCONNU : n
INCONNU : i
INCONNU : f
INCONNU : x
ENTIER : 1 (valeur : 1)
INCONNU : r
INCONNU : e
INCONNU : t
INCONNU : u
INCONNU : r
INCONNU : n
INCONNU : _
INCONNU : v
INCONNU : a
INCONNU : r
PS C:\Users\pcc\Downloads\flex>
```

Observation importante

La sortie montre des `INCONNU` lettre par lettre car `partie3.l` ne reconnaît pas les identifiants (lettres isolées). Ceci confirme l'importance de l'ordre des règles et la nécessité d'ajouter une règle identifiant dans un analyseur complet.

Sortie observée

```
$ echo "x = 3.14 + 2; // valeur" | ./partie3

INCONNU    : x
OPERATEUR  : =
FLOTTANT   : 3.14 (valeur : 3.1400)
OPERATEUR  : +
ENTIER     : 2 (valeur : 2)
DELIM      : ;
```

4.4 Réponses aux questions

Questions

1. **Flottant avant entier :** Sans cette précaution, `3.14` serait reconnu comme l'entier `3`, puis le caractère `.`, puis l'entier `14`.

2. **Signe ambigu :** `[+-]?{CHIFFRE}+` reconnaît `+3` et `-7`. Dans `a + -3`, le signe

- est absorbé dans -3 plutôt que d'être un opérateur séparé.

3. **Commentaires /* ... */ multilignes :** `"/*"([^*]|[*]+[^*/])*[*]+"/"` gère les étoiles internes sans fermer prématurément le commentaire.

5 Partie 4 : Mini-projet — Analyseur MiniLang

5.1 Spécification du langage

Spécification MiniLang

Types	`int`, `float`, `bool`
Mots-clés	`if`, `else`, `while`, `print`, `return`, `true`, `false`
Identifiants	lettres/chiffres/_, commence par lettre ou _
Littéraux	entiers, flottants, chaînes entre guillemets
Opérateurs	arithmétiques, relationnels, logiques (`&&`, `||`, `!`)
Délimiteurs	`( ) { } ; , .`
Commentaires	`//` ligne et `/* ... */` bloc

5.2 Code source — minilang.l

minilang.l

```
%{
#include <stdio.h>
#include <string.h>

int num_ligne = 1;

typedef enum {
    TK_MOT_CLE, TK_IDENT, TK_ENTIER, TK_FLOTTANT,
    TK_CHAINE, TK_OPERATEUR, TK_DELIM, TK_ERREUR
} TypeToken;

void afficher_token(TypeToken t, const char *lexeme, int ligne);
%}

%option noyywrap

LETTRE    [a-zA-Z_]
CHIFFRE   [0-9]
IDENT     {LETTRE}({LETTRE}|{CHIFFRE})*
ENTIER    {CHIFFRE}+
FLOTTANT  {CHIFFRE}+"."{CHIFFRE}+
CHAINE    \"([^\"\n])*\"
ESPACE    [ \t]+
COM_LIGNE "//"[^\n]*
COM_BLOC  "/*"([^*]|[*]+[^*/])*[*]+"/"
```

```
%%

/* 1 - Mots-cles */
"int"    { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"float"  { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"bool"   { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"if"     { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"else"   { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"while"  { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"print"  { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"return" { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"true"   { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }
"false"  { afficher_token(TK_MOT_CLE,   yytext, num_ligne); }

/* 2 - Identifiants */
{IDENT}  { afficher_token(TK_IDENT,     yytext, num_ligne); }

/* 3 - Flottants (avant entiers !) */
{FLOTTANT} { afficher_token(TK_FLOTTANT, yytext, num_ligne); }

/* 4 - Entiers */
{ENTIER} { afficher_token(TK_ENTIER,    yytext, num_ligne); }

/* 5 - Chaines */
{CHAINE} { afficher_token(TK_CHAINE,    yytext, num_ligne); }

/* 6 - Operateurs logiques */
"&&"     { afficher_token(TK_OPERATEUR, yytext, num_ligne); }
"||"     { afficher_token(TK_OPERATEUR, yytext, num_ligne); }
"!"      { afficher_token(TK_OPERATEUR, yytext, num_ligne); }

/* 7 - Operateurs relationnels */
"=="     { afficher_token(TK_OPERATEUR, yytext, num_ligne); }
"!="     { afficher_token(TK_OPERATEUR, yytext, num_ligne); }
"<="     { afficher_token(TK_OPERATEUR, yytext, num_ligne); }
">="     { afficher_token(TK_OPERATEUR, yytext, num_ligne); }
"<"      { afficher_token(TK_OPERATEUR, yytext, num_ligne); }
">"      { afficher_token(TK_OPERATEUR, yytext, num_ligne); }

/* 8 - Operateurs arithmetiques */
[+\-*/%] { afficher_token(TK_OPERATEUR, yytext, num_ligne); }

/* 9 - Affectation */
"="      { afficher_token(TK_OPERATEUR, yytext, num_ligne); }

/* 10 - Delimiteurs */
[(){};,] { afficher_token(TK_DELIM,     yytext, num_ligne); }

/* 11 - Commentaires (ignorer) */
{COM_LIGNE} { /* ignorer */ }
{COM_BLOC}  { /* ignorer */ }

/* 12 - Sauts de ligne */
\n          { num_ligne++; }
```

```
/* 13 - Espaces */
{ESPACE}     { /* ignorer */ }

/* 14 - Caracteres inconnus */
.            { printf("[ERREUR ligne %d] caractere inconnu : '%s'\n",
                        num_ligne, yytext); }

%%

void afficher_token(TypeToken t, const char *lexeme, int ligne) {
    const char *noms[] = {
        "MOT-CLE", "IDENT", "ENTIER", "FLOTTANT",
        "CHAINE", "OPERATEUR", "DELIM", "ERREUR"
    };
    printf("[ligne %2d] %-12s : %s\n", ligne, noms[t], lexeme);
}

int main(void) {
    printf("=== Analyseur Lexical MiniLang ===\n\n");
    yylex();
    printf("\nAnalyse terminee. Nombre de lignes : %d\n", num_ligne);
    return 0;
}
```

5.3 Fichier de test — test.ml

test.ml

```
// Programme de test MiniLang
int main() {
    int x = 42;
    float pi = 3.14;
    bool ok = true;

    if (x > 10) {
        print("x est grand");
    } else {
        x = x + 1;
    }

    while (x != 0) {
        x = x - 1;
    }
    return 0;
}
```

5.4 Résultat de l'analyse

Capture 10 - Compilation de minilang.l

```
C:\Users\pcc\Downloads\flex>flex minilang.l

C:\Users\pcc\Downloads\flex>gcc lex.yy.c -o minilang.exe

C:\Users\pcc\Downloads\flex>
```

Capture 11 - Sortie de l'analyseur MiniLang (début - lignes 2 à 8)

```
C:\Users\pcc\Downloads\flex>minilang.exe < test.ml
=== Analyseur Lexical MiniLang ===

[ligne  2] MOT-CLE      : int
[ligne  2] IDENT        : main
[ligne  2] DELIM        : (
[ligne  2] DELIM        : )
[ligne  2] DELIM        : {
[ligne  3] MOT-CLE      : int
[ligne  3] IDENT        : x
[ligne  3] OPERATEUR    : =
[ligne  3] ENTIER       : 42
[ligne  3] DELIM        : ;
[ligne  4] MOT-CLE      : float
[ligne  4] IDENT        : pi
[ligne  4] OPERATEUR    : =
[ligne  4] FLOTTANT     : 3.14
[ligne  4] DELIM        : ;
[ligne  5] MOT-CLE      : bool
[ligne  5] IDENT        : ok
[ligne  5] OPERATEUR    : =
[ligne  5] MOT-CLE      : true
[ligne  5] DELIM        : ;
[ligne  7] MOT-CLE      : if
[ligne  7] DELIM        : (
[ligne  7] IDENT        : x
[ligne  7] OPERATEUR    : >
[ligne  7] ENTIER       : 10
[ligne  7] DELIM        : )
[ligne  7] DELIM        : {
[ligne  8] MOT-CLE      : print
[ligne  8] DELIM        : (
```

Capture 12 - Sortie de l'analyseur MiniLang (fin - lignes 8 à 17)

```
[ligne  8] CHAINE       : "x est grand"
[ligne  8] DELIM        : )
[ligne  8] DELIM        : ;
[ligne  9] DELIM        : }
[ligne  9] MOT-CLE      : else
[ligne  9] DELIM        : {
[ligne 10] IDENT        : x
[ligne 10] OPERATEUR    : =
[ligne 10] IDENT        : x
[ligne 10] OPERATEUR    : +
[ligne 10] ENTIER       : 1
[ligne 10] DELIM        : ;
[ligne 11] DELIM        : }
[ligne 13] MOT-CLE      : while
[ligne 13] DELIM        : (
[ligne 13] IDENT        : x
[ligne 13] OPERATEUR    : !=
[ligne 13] ENTIER       : 0
[ligne 13] DELIM        : )
[ligne 13] DELIM        : {
[ligne 14] IDENT        : x
[ligne 14] OPERATEUR    : =
[ligne 14] IDENT        : x
[ligne 14] OPERATEUR    : -
[ligne 14] ENTIER       : 1
[ligne 14] DELIM        : ;
[ligne 15] DELIM        : }
[ligne 16] MOT-CLE      : return
[ligne 16] ENTIER       : 0
[ligne 16] DELIM        : ;
[ligne 17] DELIM        : }

Analyse terminee. Nombre de lignes : 17
```

Sortie observée

```
=== Analyseur Lexical MiniLang ===

[ligne  2] MOT-CLE      : int
[ligne  2] IDENT        : main
[ligne  2] DELIM        : (
[ligne  2] DELIM        : )
[ligne  2] DELIM        : {
[ligne  3] MOT-CLE      : int
[ligne  3] IDENT        : x
[ligne  3] OPERATEUR    : =
[ligne  3] ENTIER       : 42
[ligne  3] DELIM        : ;
[ligne  4] MOT-CLE      : float
[ligne  4] IDENT        : pi
[ligne  4] OPERATEUR    : =
[ligne  4] FLOTTANT     : 3.14
...
Analyse terminee. Nombre de lignes : 17
```

5.5 Tokens de `float res = (pi * 2.0) + x;`

Lexème	Token	Règle
`float`	**MOT-CLE**	chaîne littérale `"float"`
`res`	**IDENT**	`{IDENT}`
`=`	**OPERATEUR**	affectation
`(`	**DELIM**	délimiteur
`pi`	**IDENT**	`{IDENT}`
	OPERATEUR	opérateur arithmétique
`2.0`	**FLOTTANT**	`{FLOTTANT}`
`)`	**DELIM**	délimiteur
`+`	**OPERATEUR**	opérateur arithmétique
`x`	**IDENT**	`{IDENT}`
`;`	**DELIM**	délimiteur

5.6 Réponses aux questions

? Questions

1. **Mise à jour de `num_ligne` :** La règle `\n { num_ligne++; }` l'incrémente à chaque saut de ligne. Pour un commentaire bloc multi-lignes, les `\n` absorbés par `COM_BLOC` ne sont pas comptés : il faut parcourir `yytext` pour les comptabiliser manuellement.

2. **Commentaires bloc avec comptage des lignes :**

```
{COM_BLOC} { int i;
             for(i=0; yytext[i]; i++)
               if(yytext[i]=='\n') num_ligne++; }
```

3. **Transmission à Bison :** Inclure `parser.tab.h`, utiliser `yylval` pour la valeur sémantique, et remplacer `printf` par `return TOKEN_CODE` dans chaque règle.

4. **Tokens de `float res = (pi * 2.0) + x;` :** 11 tokens produits — voir tableau ci-dessus.

6 Conclusion

- Structure en trois sections d'un fichier `.l`.
- Écriture d'expressions régulières pour toutes les catégories de tokens.
- Importance de l'ordre des règles : mots-clés avant identifiants, flottants avant entiers.
- Utilisation de `yytext`, `yyleng` et gestion du compteur de lignes.
- Flux de compilation complet : `flex` → `gcc` → exécution.

L'analyseur lexical de la Partie 4 constitue la base du **TP 3** (analyse syntaxique avec Bison), où les tokens produits ici seront consommés par un analyseur syntaxique.

6 Annexe — Récapitulatif des commandes Flex

Commande / Élément	Rôle
`flex fichier.l`	Génère `lex.yy.c`
`gcc lex.yy.c -o prog -lfl`	Compile l'analyseur (Linux/macOS)
`gcc lex.yy.c -o prog`	Compile sans bibliothèque (avec `noyywrap`)
`yytext`	Pointeur vers le lexème reconnu
`yyleng`	Longueur du lexème
`yylex()`	Lance l'analyse lexicale
`yywrap()`	Appelée en fin de fichier
`ECHO`	Réécrit `yytext` sur la sortie
`%option noyywrap`	Supprime `yywrap`, simplifie la compilation
`%option yylineno`	Active le compteur de lignes automatique
`%option case-insensitive`	Analyse insensible à la casse